# AlgoFlow

*One page project summary*

Submitted by Krish Sondagar | 15 October 2026

**Source repository** https://github.com/KRISH141006/AlgoFlow

## Problem statement

Solving a DSA problem once does not ensure that its approach will be remembered. Accepted code, topic notes, and revision dates are often scattered across coding platforms and local files. AlgoFlow brings these records into one local workspace and schedules deliberate revision.

## Objectives

Organize accepted LeetCode and GeeksforGeeks solutions; preserve exact code and problem descriptions; schedule problem and theory reviews; record improvements as solution versions; and support safe backup and restoration.

## Key features

Single and batch JSON import with duplicate protection; searchable library with difficulty, platform, and topic filters; sanitized problem descriptions; C++, C, and SQL code viewing; explicit review completion; immutable solution snapshots with self-assessment; progress statistics, activity heatmap, and additive backup restore.

## Technologies and tools

Python, FastAPI, Pydantic, SQLite, React, TypeScript, Vite, Uvicorn, Bleach, pytest, and GitHub. The application uses FastAPI rather than Django.

## Implementation

The React interface calls versioned FastAPI endpoints. Pydantic validates form submissions, and service functions normalize exported platform data, identify duplicate solutions, and manage review stages. SQLite transactions store problems, topics, versions, and review histories. The revision schedule uses intervals of 1, 7, 30, and 180 days. Separate personal and demonstration databases protect original records during presentation.

## Achieved outcome

The verified snapshot contains 274 accepted solutions with code and descriptions. One preserved tracker problem brings the library to 275 problems; the Sliding Window topic and its review history are retained. All 13 automated tests pass. Browser checks verified import, search, saved versions, review completion, restart persistence, and backup download and restoration. Historical imports do not invent solve dates or activity. Account management, live extension delivery, and cloud synchronization remain future work.